In [1]:
```
# ==========================================================
# 2. TEXT CLASSIFICATION NOTEBOOK
# ==========================================================

# ------------------------------------------------
# 2.1 IMPORTS AND SETUP
# ------------------------------------------------
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from textblob import TextBlob
import warnings
warnings.filterwarnings('ignore')

# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

print("Libraries imported successfully")

# ------------------------------------------------
# 2.2 LOAD AND PREPROCESS DATA
# ------------------------------------------------
print("\nLoading dataset...")
# Use relative path to locate the dataset file
data_path = "data/overview-of-recordings.csv"
data = pd.read_csv(data_path)
print(f"Dataset shape: {data.shape}")
print(f"Number of unique prompts: {data['prompt'].nunique()}")
print(f"Prompt classes: {data['prompt'].unique()}")

# Check for class imbalance
print("\nClass distribution:")
class_distribution = data['prompt'].value_counts()
print(class_distribution)

# Plot class distribution
plt.figure(figsize=(10, 6))
sns.barplot(x=class_distribution.values, y=class_distribution.index)
```

```
plt.title("Distribution of Medical Prompt Classes")
plt.xlabel("Number of Samples")
plt.tight_layout()
plt.show()

# Text preprocessing function with lemmatization
def preprocess_text(text):
    if pd.isna(text):
        return ""

    # Lowercase
    text = text.lower()

    # Remove special characters
    text = ''.join([c for c in text if c.isalnum() or c.isspace()])

    # Tokenize
    tokens = word_tokenize(text)

    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    tokens = [token for token in tokens if token not in stop_words]

    # Lemmatize
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(token) for token in tokens]

    # Join back to string
    return ' '.join(tokens)

# Apply preprocessing
print("\nPreprocessing text data...")
data['processed_text'] = data['phrase'].apply(preprocess_text)

# Display a sample of processed text
print("\nSample of processed text:")
for i, row in data.sample(3).iterrows():
    print(f"Original: {row['phrase']}")
    print(f"Processed: {row['processed_text']}")
    print()

# ----------------------------------------------
# 2.3 FEATURE EXPLORATION
# ----------------------------------------------
# Extract additional text features
print("\nExtracting additional text features...")
data['text_length'] = data['processed_text'].apply(len)
data['word_count'] = data['processed_text'].apply(lambda x: len(x.split()))
data['sentiment'] = data['phrase'].apply(lambda x: TextBlob(str(x)).sentiment.polar
data['subjectivity'] = data['phrase'].apply(lambda x: TextBlob(str(x)).sentiment.su

# Plot additional feature distributions
plt.figure(figsize=(16, 12))

plt.subplot(2, 2, 1)
sns.boxplot(x='prompt', y='text_length', data=data)
```

```
plt.title("Text Length by Prompt")
plt.xticks(rotation=45)

plt.subplot(2, 2, 2)
sns.boxplot(x='prompt', y='word_count', data=data)
plt.title("Word Count by Prompt")
plt.xticks(rotation=45)

plt.subplot(2, 2, 3)
sns.boxplot(x='prompt', y='sentiment', data=data)
plt.title("Sentiment by Prompt")
plt.xticks(rotation=45)

plt.subplot(2, 2, 4)
sns.boxplot(x='prompt', y='subjectivity', data=data)
plt.title("Subjectivity by Prompt")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# ------------------------------------------------
# 2.4 DATA SPLITTING FOR TRAINING AND TESTING
# ------------------------------------------------
print("\nSplitting data into training and testing sets...")
# Extract the core features and target
X_text = data['processed_text']
X_additional = data[['text_length', 'word_count', 'sentiment', 'subjectivity']]
y = data['prompt']

# Split the data with stratification
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)

X_train_add, X_test_add, _, _ = train_test_split(
    X_additional, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set size: {len(X_train_text)}")
print(f"Testing set size: {len(X_test_text)}")

# ------------------------------------------------
# 2.5 MODEL BUILDING AND EVALUATION - TF-IDF BASED
# ------------------------------------------------
print("\n--- TF-IDF BASED MODELS ---")

# Define TF-IDF pipeline
tfidf_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
])

# Transform training and testing data
X_train_tfidf = tfidf_pipeline.fit_transform(X_train_text)
X_test_tfidf = tfidf_pipeline.transform(X_test_text)
```

```
print(f"TF-IDF features shape: {X_train_tfidf.shape}")

# 2.5.1 Logistic Regression with TF-IDF
print("\nTraining Logistic Regression model...")
lr_model = LogisticRegression(max_iter=1000, C=10, solver='liblinear', multi_class=
lr_model.fit(X_train_tfidf, y_train)
lr_predictions = lr_model.predict(X_test_tfidf)

print("\nLogistic Regression Results:")
lr_accuracy = accuracy_score(y_test, lr_predictions)
print(f"Accuracy: {lr_accuracy:.4f}")
print(classification_report(y_test, lr_predictions))

# 2.5.2 Random Forest with TF-IDF
print("\nTraining Random Forest model...")
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_tfidf, y_train)
rf_predictions = rf_model.predict(X_test_tfidf)

print("\nRandom Forest Results:")
rf_accuracy = accuracy_score(y_test, rf_predictions)
print(f"Accuracy: {rf_accuracy:.4f}")
print(classification_report(y_test, rf_predictions))

# 2.5.3 Support Vector Machine with TF-IDF
print("\nTraining SVM model...")
svm_model = LinearSVC(C=1.0, random_state=42, max_iter=10000)
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)

print("\nSVM Results:")
svm_accuracy = accuracy_score(y_test, svm_predictions)
print(f"Accuracy: {svm_accuracy:.4f}")
print(classification_report(y_test, svm_predictions))

# 2.5.4 Naive Bayes with TF-IDF
print("\nTraining Naive Bayes model...")
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_predictions = nb_model.predict(X_test_tfidf)

print("\nNaive Bayes Results:")
nb_accuracy = accuracy_score(y_test, nb_predictions)
print(f"Accuracy: {nb_accuracy:.4f}")
print(classification_report(y_test, nb_predictions))

# ----------------------------------------------
# 2.6 CONFUSION MATRIX ANALYSIS
# ----------------------------------------------
# Identify the best performing model
print("\nIdentifying the best performing model...")
accuracies = {
    'Logistic Regression': lr_accuracy,
    'Random Forest': rf_accuracy,
    'SVM': svm_accuracy,
    'Naive Bayes': nb_accuracy
```

```
}

best_model_name = max(accuracies, key=accuracies.get)
best_predictions = None

if best_model_name == 'Logistic Regression':
    best_predictions = lr_predictions
elif best_model_name == 'Random Forest':
    best_predictions = rf_predictions
elif best_model_name == 'SVM':
    best_predictions = svm_predictions
else:
    best_predictions = nb_predictions

print(f"\nBest model: {best_model_name} with accuracy: {accuracies[best_model_name]

# Plot confusion matrix for the best model
plt.figure(figsize=(14, 12))
cm = confusion_matrix(y_test, best_predictions)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data['prompt'].unique(),
            yticklabels=data['prompt'].unique())
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title(f'Confusion Matrix: {best_model_name}')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# ----------------------------------------------
# 2.7 FEATURE IMPORTANCE ANALYSIS
# ----------------------------------------------
print("\nAnalyzing feature importance...")

# Get the TF-IDF vectorizer
tfidf_vectorizer = tfidf_pipeline.named_steps['tfidf']
feature_names = tfidf_vectorizer.get_feature_names_out()

# For each class, get top features (for Logistic Regression or SVM)
if best_model_name in ['Logistic Regression', 'SVM']:
    model = lr_model if best_model_name == 'Logistic Regression' else svm_model

    # For multiclass, we have a coefficient vector per class
    if hasattr(model, 'coef_'):
        for i, class_label in enumerate(model.classes_):
            # Get top 15 features for this class
            if model.coef_.shape[0] > i:  # Ensure we don't exceed array bounds
                top_features_idx = np.argsort(model.coef_[i])[-15:]
                top_features = [feature_names[idx] for idx in top_features_idx]
                print(f"\nTop features for '{class_label}':")
                print(", ".join(top_features))

# For Random Forest, feature importance is different
elif best_model_name == 'Random Forest':
    feature_importances = rf_model.feature_importances_
    indices = np.argsort(feature_importances)[-20:]  # Top 20 features
```

```
    plt.figure(figsize=(10, 8))
    plt.title('Top 20 Feature Importances')
    plt.barh(range(len(indices)), feature_importances[indices], color='b', align='c
    plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
    plt.xlabel('Relative Importance')
    plt.tight_layout()
    plt.show()

# ------------------------------------------------
# 2.8 CROSS-VALIDATION FOR MODEL ROBUSTNESS
# ------------------------------------------------
print("\nPerforming cross-validation for model robustness...")

# Create a combined TF-IDF + classifier pipeline for each model
lr_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
    ('classifier', LogisticRegression(max_iter=1000, C=10, solver='liblinear', mult
])

rf_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
])

svm_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
    ('classifier', LinearSVC(C=1.0, random_state=42, max_iter=10000))
])

nb_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
    ('classifier', MultinomialNB())
])

# Define stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Perform cross-validation for each model
print("\nLogistic Regression cross-validation:")
lr_cv_scores = cross_val_score(lr_pipeline, X_text, y, cv=cv, scoring='accuracy')
print(f"Mean CV Score: {lr_cv_scores.mean():.4f}, Std: {lr_cv_scores.std():.4f}")
print(f"Individual scores: {lr_cv_scores}")

print("\nRandom Forest cross-validation:")
rf_cv_scores = cross_val_score(rf_pipeline, X_text, y, cv=cv, scoring='accuracy')
print(f"Mean CV Score: {rf_cv_scores.mean():.4f}, Std: {rf_cv_scores.std():.4f}")
print(f"Individual scores: {rf_cv_scores}")

print("\nSVM cross-validation:")
svm_cv_scores = cross_val_score(svm_pipeline, X_text, y, cv=cv, scoring='accuracy')
print(f"Mean CV Score: {svm_cv_scores.mean():.4f}, Std: {svm_cv_scores.std():.4f}")
print(f"Individual scores: {svm_cv_scores}")

print("\nNaive Bayes cross-validation:")
nb_cv_scores = cross_val_score(nb_pipeline, X_text, y, cv=cv, scoring='accuracy')
```

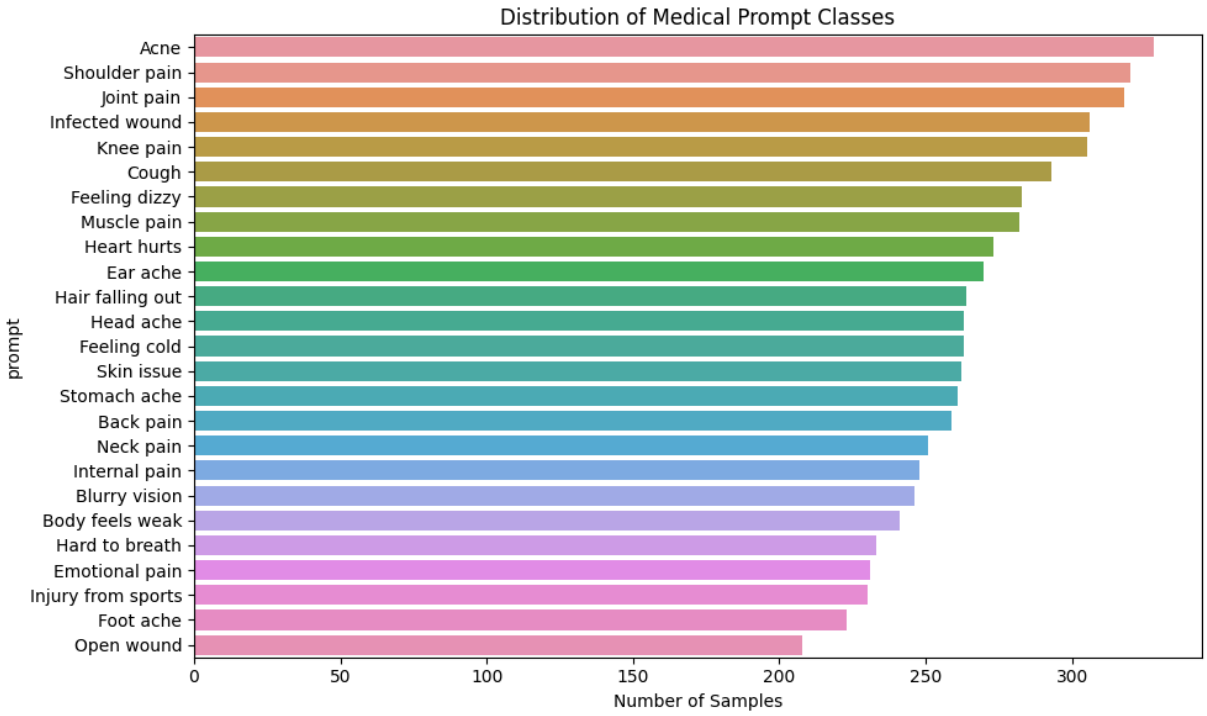
```
print(f"Mean CV Score: {nb_cv_scores.mean():.4f}, Std: {nb_cv_scores.std():.4f}")
print(f"Individual scores: {nb_cv_scores}")

# Compare CV results
cv_results = {
    'Logistic Regression': lr_cv_scores.mean(),
    'Random Forest': rf_cv_scores.mean(),
    'SVM': svm_cv_scores.mean(),
    'Naive Bayes': nb_cv_scores.mean()
}

# Plot cross-validation results
plt.figure(figsize=(10, 6))
models = list(cv_results.keys())
scores = list(cv_results.values())
plt.bar(models, scores)
plt.ylim(0, 1.0)
plt.ylabel('Mean Accuracy')
plt.title('Cross-Validation Results Comparison')
plt.xticks(rotation=45)
for i, v in enumerate(scores):
    plt.text(i, v + 0.02, f"{v:.4f}", ha='center')
plt.tight_layout()
plt.show()

# ------------------------------------------------
# 2.9 ENHANCED MODEL WITH ADDITIONAL FEATURES
# ------------------------------------------------
print("\n--- ENHANCED MODEL WITH ADDITIONAL FEATURES ---")

# Combine TF-IDF features with additional text features
from scipy.sparse import hstack

# Get numerical features scaled
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_add_scaled = scaler.fit_transform(X_train_add)
X_test_add_scaled = scaler.transform(X_test_add)

# Combine with TF-IDF features
X_train_combined = hstack([X_train_tfidf, X_train_add_scaled])
X_test_combined = hstack([X_test_tfidf, X_test_add_scaled])

print(f"Combined features shape: {X_train_combined.shape}")

# Train the best model on combined features
if best_model_name == 'Logistic Regression':
    enhanced_model = LogisticRegression(max_iter=1000, C=10, solver='liblinear', mu
elif best_model_name == 'Random Forest':
    enhanced_model = RandomForestClassifier(n_estimators=100, random_state=42)
elif best_model_name == 'SVM':
    enhanced_model = LinearSVC(C=1.0, random_state=42, max_iter=10000)
else:
    enhanced_model = MultinomialNB()

print(f"\nTraining enhanced {best_model_name} model with combined features...")
```

```
enhanced_model.fit(X_train_combined, y_train)
enhanced_predictions = enhanced_model.predict(X_test_combined)

print("\nEnhanced Model Results:")
enhanced_accuracy = accuracy_score(y_test, enhanced_predictions)
print(f"Accuracy: {enhanced_accuracy:.4f}")
print(classification_report(y_test, enhanced_predictions))

# Compare with the best TF-IDF only model
print(f"\nComparison:")
print(f"Best TF-IDF model ({best_model_name}): {accuracies[best_model_name]:.4f}")
print(f"Enhanced model with additional features: {enhanced_accuracy:.4f}")
improvement = enhanced_accuracy - accuracies[best_model_name]
print(f"Improvement: {improvement:.4f} ({improvement*100:.2f}%)")

# Plot confusion matrix for the enhanced model
plt.figure(figsize=(14, 12))
cm_enhanced = confusion_matrix(y_test, enhanced_predictions)
sns.heatmap(cm_enhanced, annot=True, fmt='d', cmap='Blues',
            xticklabels=data['prompt'].unique(),
            yticklabels=data['prompt'].unique())
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix: Enhanced Model')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# ------------------------------------------------
# 2.10 MODEL SAVING
# ------------------------------------------------
print("\nSaving the best model...")
import pickle

# Ensure the models directory exists
os.makedirs('models', exist_ok=True)

# Create a full pipeline for the best model including preprocessing
best_pipeline = None

if best_model_name == 'Logistic Regression':
    best_pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('classifier', LogisticRegression(max_iter=1000, C=10, solver='liblinear',
    ])
elif best_model_name == 'Random Forest':
    best_pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
    ])
elif best_model_name == 'SVM':
    best_pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('classifier', LinearSVC(C=1.0, random_state=42, max_iter=10000))
    ])
else:
```

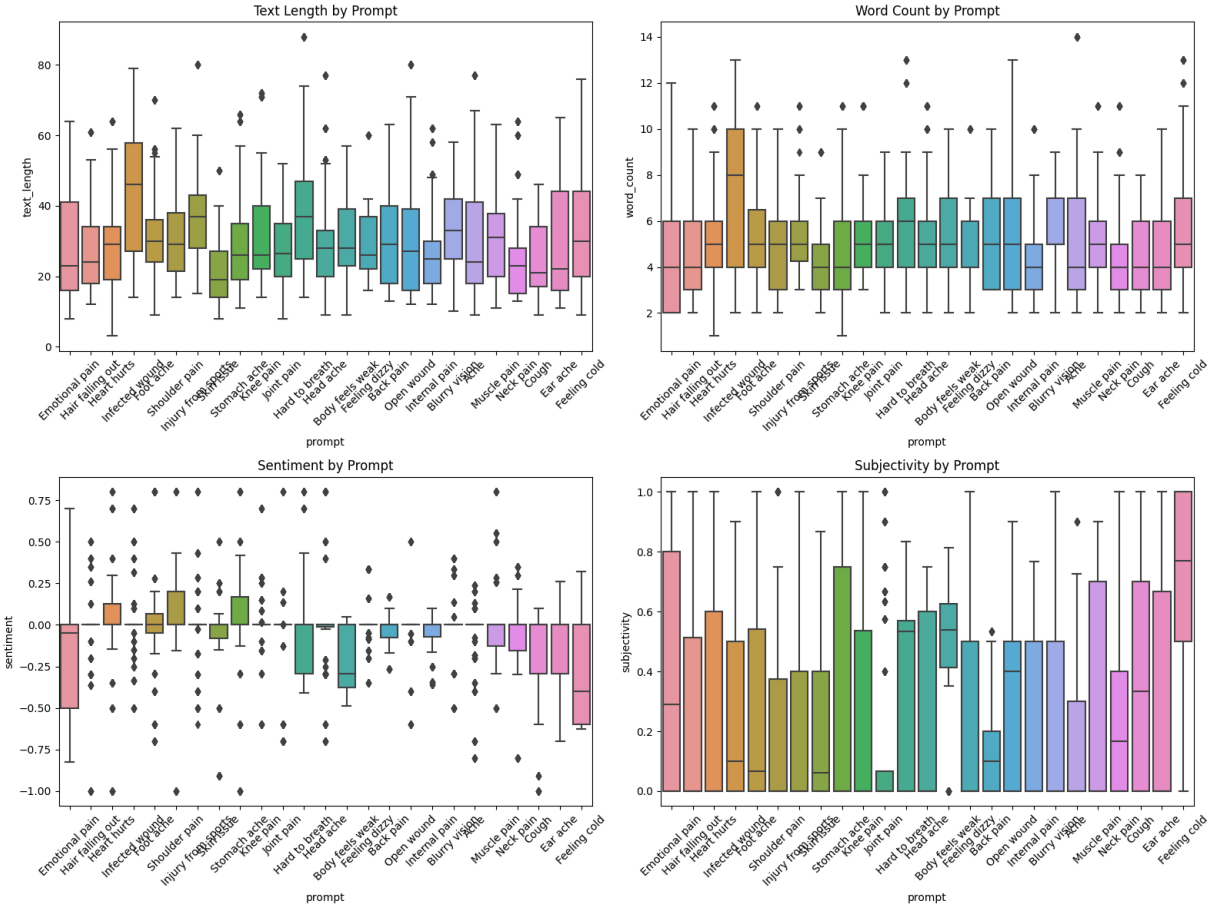
```
    best_pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('classifier', MultinomialNB())
    ])

# Fit the pipeline on all text data
best_pipeline.fit(X_text, y)

# Save the pipeline
with open('models/text_classification_model.pkl', 'wb') as f:
    pickle.dump(best_pipeline, f)
print("Model saved as 'models/text_classification_model.pkl'")

# Save the enhanced model pipeline if it performed better
if enhanced_accuracy > accuracies[best_model_name]:
    # Note about saving enhanced model
    print("\nNote: The enhanced model with additional features requires custom")
    print("preprocessing steps not fully captured in this pipeline.")
    print("For production use, you would need to create a custom transformer")
    print("to handle both text and additional feature extraction.")

# ------------------------------------------------
# 2.11 SUMMARY AND CONCLUSIONS
# ------------------------------------------------
print("\n--- SUMMARY AND CONCLUSIONS ---")
print("1. The text classification analysis was performed on multiple medical prompt
print(f"2. The best performing model was {best_model_name} with accuracy: {accuraci
print(f"3. Cross-validation showed a robust performance with mean accuracy: {cv_res

if enhanced_accuracy > accuracies[best_model_name]:
    print("4. Incorporating additional features (text length, sentiment, etc.) impr
    print(f"   Enhanced model accuracy: {enhanced_accuracy:.4f} (improvement: {impr
else:
    print("4. Incorporating additional features did not significantly improve perfo

print("5. The trained model can effectively classify medical prompts based on text
print("6. Key findings from the confusion matrix show which prompt categories are m
print("7. Research objective of performing multi-class classification of medical pr
```

```
[nltk_data] Downloading package punkt to C:\Users\S-C-R-E-
[nltk_data]     A-M\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to C:\Users\S-C-R-E-
[nltk_data]     A-M\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\S-C-R-E-
[nltk_data]     A-M\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
Libraries imported successfully

Loading dataset...
Dataset shape: (6661, 13)
Number of unique prompts: 25
Prompt classes: ['Emotional pain' 'Hair falling out' 'Heart hurts' 'Infected wound'
 'Foot ache' 'Shoulder pain' 'Injury from sports' 'Skin issue'
 'Stomach ache' 'Knee pain' 'Joint pain' 'Hard to breath' 'Head ache'
 'Body feels weak' 'Feeling dizzy' 'Back pain' 'Open wound'
 'Internal pain' 'Blurry vision' 'Acne' 'Muscle pain' 'Neck pain' 'Cough'
 'Ear ache' 'Feeling cold']

Class distribution:
prompt
Acne                  328
Shoulder pain         320
Joint pain            318
Infected wound        306
Knee pain             305
Cough                 293
Feeling dizzy         283
Muscle pain           282
Heart hurts           273
Ear ache              270
Hair falling out      264
Head ache             263
Feeling cold          263
Skin issue            262
Stomach ache          261
Back pain             259
Neck pain             251
Internal pain         248
Blurry vision         246
Body feels weak       241
Hard to breath        233
Emotional pain        231
Injury from sports    230
Foot ache             223
Open wound            208
Name: count, dtype: int64
```



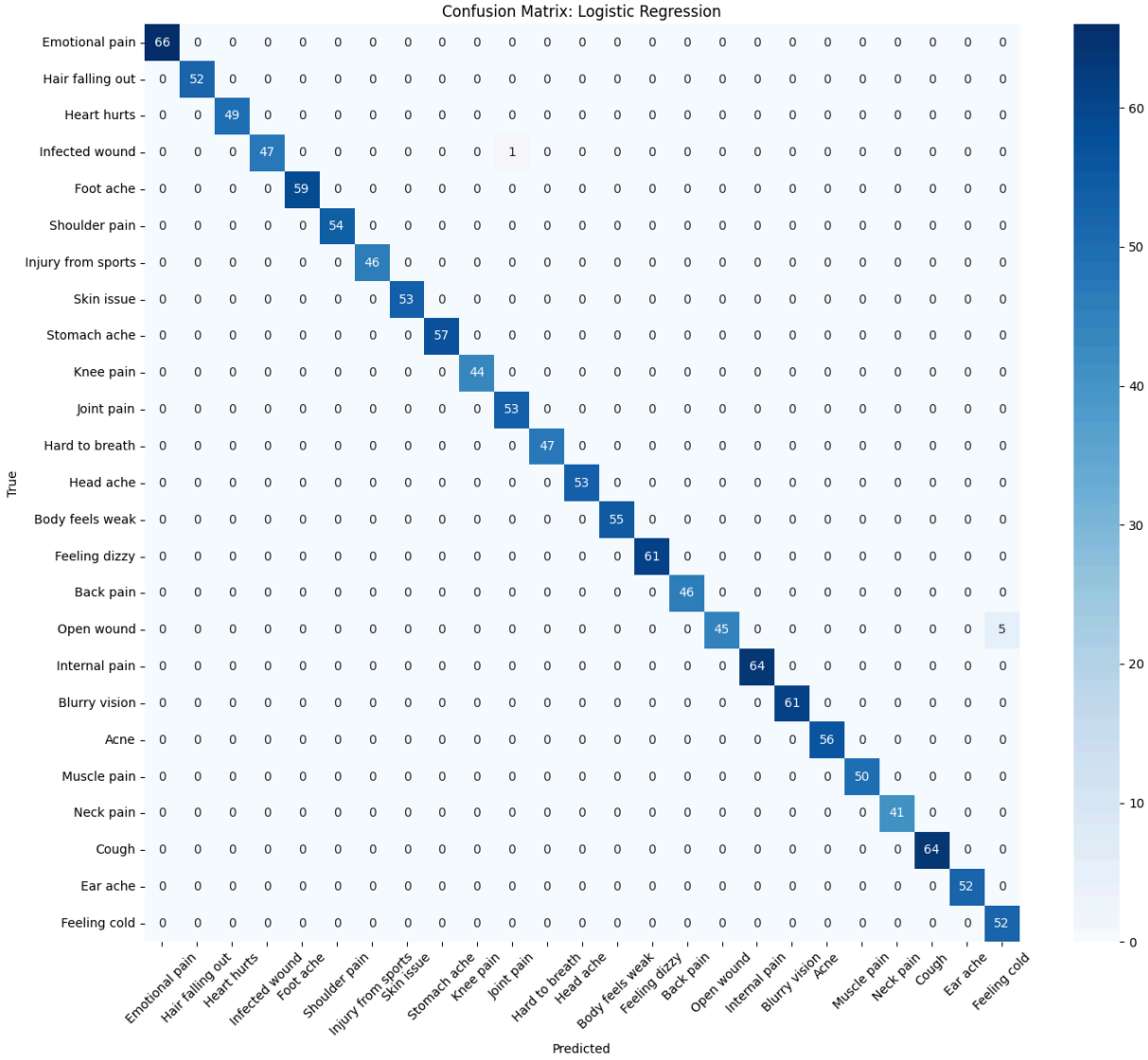

```
Preprocessing text data...

Sample of processed text:
Original: I could not exercise because of my knee pain
Processed: could exercise knee pain

Original: i feel dizzy after severe diarrhea
Processed: feel dizzy severe diarrhea

Original: surgical wounds red, firing pus, painful and hot to the touch
Processed: surgical wound red firing pu painful hot touch


Extracting additional text features...
```


Text Length by Prompt
text_length
prompt
Word Count by Prompt
word_count
prompt
Sentiment by Prompt
sentiment
prompt
Subjectivity by Prompt
subjectivity
prompt
Emotional pain
Hair falling out
Heart hurts
Infected wound
Foot ache
Shoulder pain
Injury from sports
Skin rash
Stomach ache
Knee pain
Joint pain
Hard to breath
Head ache
Body feels weak
Feeling dizzy
Back pain
Open wound
Internal pain
Blurry vision
Acne
Muscle pain
Neck pain
Cough
Ear ache
Feeling cold

```
Splitting data into training and testing sets...
Training set size: 5328
Testing set size: 1333

--- TF-IDF BASED MODELS ---
TF-IDF features shape: (5328, 3318)

Training Logistic Regression model...

Logistic Regression Results:
Accuracy: 0.9955
                    precision    recall  f1-score   support

              Acne       1.00      1.00      1.00        66
         Back pain       1.00      1.00      1.00        52
     Blurry vision       1.00      1.00      1.00        49
   Body feels weak       1.00      0.98      0.99        48
             Cough       1.00      1.00      1.00        59
          Ear ache       1.00      1.00      1.00        54
    Emotional pain       1.00      1.00      1.00        46
      Feeling cold       1.00      1.00      1.00        53
     Feeling dizzy       1.00      1.00      1.00        57
         Foot ache       1.00      1.00      1.00        44
  Hair falling out       0.98      1.00      0.99        53
    Hard to breath       1.00      1.00      1.00        47
         Head ache       1.00      1.00      1.00        53
       Heart hurts       1.00      1.00      1.00        55
    Infected wound       1.00      1.00      1.00        61
Injury from sports       1.00      1.00      1.00        46
     Internal pain       1.00      0.90      0.95        50
        Joint pain       1.00      1.00      1.00        64
         Knee pain       1.00      1.00      1.00        61
       Muscle pain       1.00      1.00      1.00        56
         Neck pain       1.00      1.00      1.00        50
        Open wound       1.00      1.00      1.00        41
     Shoulder pain       1.00      1.00      1.00        64
        Skin issue       1.00      1.00      1.00        52
      Stomach ache       0.91      1.00      0.95        52

          accuracy                           1.00      1333
         macro avg       1.00      1.00      1.00      1333
      weighted avg       1.00      1.00      1.00      1333


Training Random Forest model...

Random Forest Results:
Accuracy: 0.9955
                    precision    recall  f1-score   support

              Acne       1.00      1.00      1.00        66
         Back pain       1.00      1.00      1.00        52
     Blurry vision       1.00      1.00      1.00        49
   Body feels weak       1.00      0.98      0.99        48
             Cough       1.00      1.00      1.00        59
          Ear ache       1.00      1.00      1.00        54
```

```
    Emotional pain       1.00      1.00      1.00        46
      Feeling cold       1.00      1.00      1.00        53
     Feeling dizzy       1.00      1.00      1.00        57
         Foot ache       1.00      1.00      1.00        44
  Hair falling out       0.98      1.00      0.99        53
    Hard to breath       1.00      1.00      1.00        47
         Head ache       1.00      1.00      1.00        53
       Heart hurts       1.00      1.00      1.00        55
    Infected wound       1.00      1.00      1.00        61
Injury from sports       1.00      1.00      1.00        46
     Internal pain       1.00      0.90      0.95        50
        Joint pain       1.00      1.00      1.00        64
         Knee pain       1.00      1.00      1.00        61
       Muscle pain       1.00      1.00      1.00        56
         Neck pain       1.00      1.00      1.00        50
        Open wound       1.00      1.00      1.00        41
     Shoulder pain       1.00      1.00      1.00        64
        Skin issue       1.00      1.00      1.00        52
      Stomach ache       0.91      1.00      0.95        52

          accuracy                           1.00      1333
         macro avg       1.00      1.00      1.00      1333
      weighted avg       1.00      1.00      1.00      1333


Training SVM model...

SVM Results:
Accuracy: 0.9955
                    precision    recall  f1-score   support

              Acne       1.00      1.00      1.00        66
         Back pain       1.00      1.00      1.00        52
     Blurry vision       1.00      1.00      1.00        49
   Body feels weak       1.00      0.98      0.99        48
             Cough       1.00      1.00      1.00        59
          Ear ache       1.00      1.00      1.00        54
    Emotional pain       1.00      1.00      1.00        46
      Feeling cold       1.00      1.00      1.00        53
     Feeling dizzy       1.00      1.00      1.00        57
         Foot ache       1.00      1.00      1.00        44
  Hair falling out       0.98      1.00      0.99        53
    Hard to breath       1.00      1.00      1.00        47
         Head ache       1.00      1.00      1.00        53
       Heart hurts       1.00      1.00      1.00        55
    Infected wound       1.00      1.00      1.00        61
Injury from sports       1.00      1.00      1.00        46
     Internal pain       1.00      0.90      0.95        50
        Joint pain       1.00      1.00      1.00        64
         Knee pain       1.00      1.00      1.00        61
       Muscle pain       1.00      1.00      1.00        56
         Neck pain       1.00      1.00      1.00        50
        Open wound       1.00      1.00      1.00        41
     Shoulder pain       1.00      1.00      1.00        64
        Skin issue       1.00      1.00      1.00        52
      Stomach ache       0.91      1.00      0.95        52
```

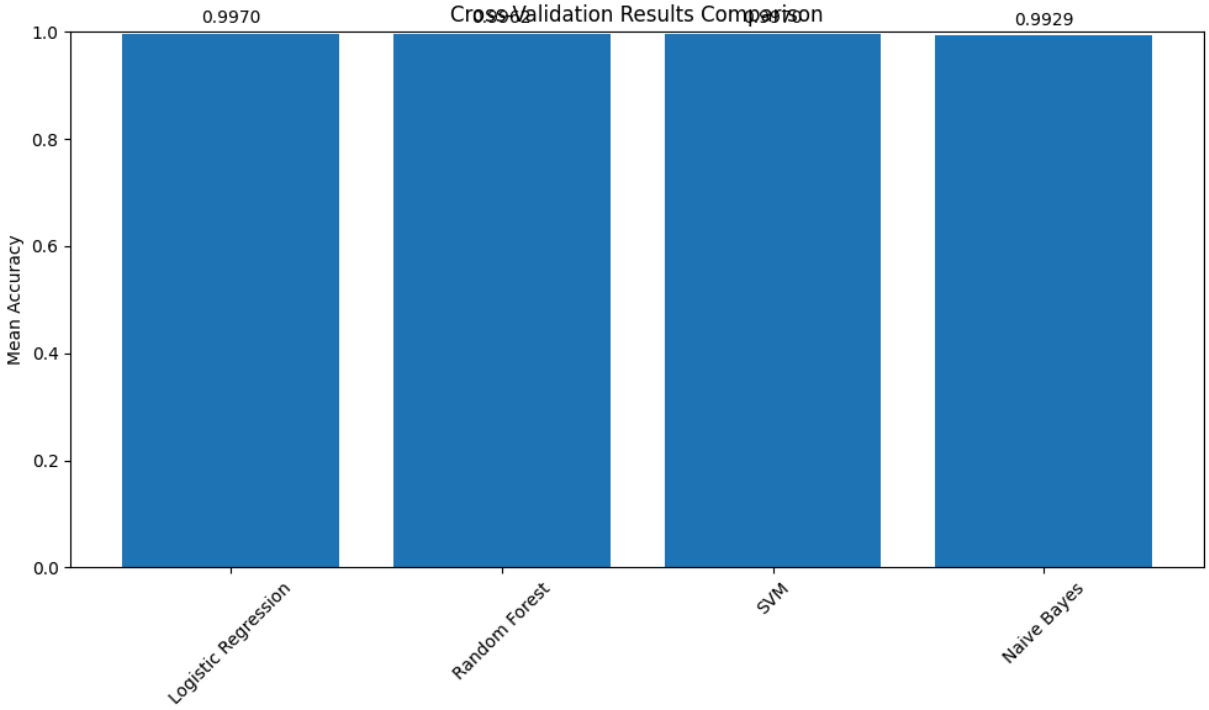
```
           accuracy                           1.00      1333
          macro avg       1.00      1.00      1.00      1333
       weighted avg       1.00      1.00      1.00      1333


Training Naive Bayes model...

Naive Bayes Results:
Accuracy: 0.9940
                    precision    recall  f1-score   support

              Acne       1.00      1.00      1.00        66
         Back pain       1.00      1.00      1.00        52
     Blurry vision       1.00      1.00      1.00        49
   Body feels weak       1.00      0.98      0.99        48
             Cough       1.00      1.00      1.00        59
          Ear ache       1.00      1.00      1.00        54
    Emotional pain       1.00      0.98      0.99        46
      Feeling cold       1.00      1.00      1.00        53
     Feeling dizzy       1.00      1.00      1.00        57
         Foot ache       1.00      1.00      1.00        44
  Hair falling out       0.98      1.00      0.99        53
    Hard to breath       1.00      1.00      1.00        47
         Head ache       1.00      1.00      1.00        53
       Heart hurts       0.98      1.00      0.99        55
    Infected wound       1.00      1.00      1.00        61
Injury from sports       1.00      1.00      1.00        46
     Internal pain       1.00      0.88      0.94        50
        Joint pain       1.00      1.00      1.00        64
         Knee pain       1.00      1.00      1.00        61
       Muscle pain       1.00      1.00      1.00        56
         Neck pain       1.00      1.00      1.00        50
        Open wound       1.00      1.00      1.00        41
     Shoulder pain       1.00      1.00      1.00        64
        Skin issue       1.00      1.00      1.00        52
      Stomach ache       0.90      1.00      0.95        52

           accuracy                           0.99      1333
          macro avg       0.99      0.99      0.99      1333
       weighted avg       0.99      0.99      0.99      1333


Identifying the best performing model...

Best model: Logistic Regression with accuracy: 0.9955
```


Confusion Matrix: Logistic Regression
True
Predicted
Emotional pain
Hair falling out
Heart hurts
Infected wound
Foot ache
Shoulder pain
Injury from sports
Skin issue
Stomach ache
Knee pain
Joint pain
Hard to breath
Head ache
Body feels weak
Feeling dizzy
Back pain
Open wound
Internal pain
Blurry vision
Acne
Muscle pain
Neck pain
Cough
Ear ache
Feeling cold
0
10
20
30
40
50
60

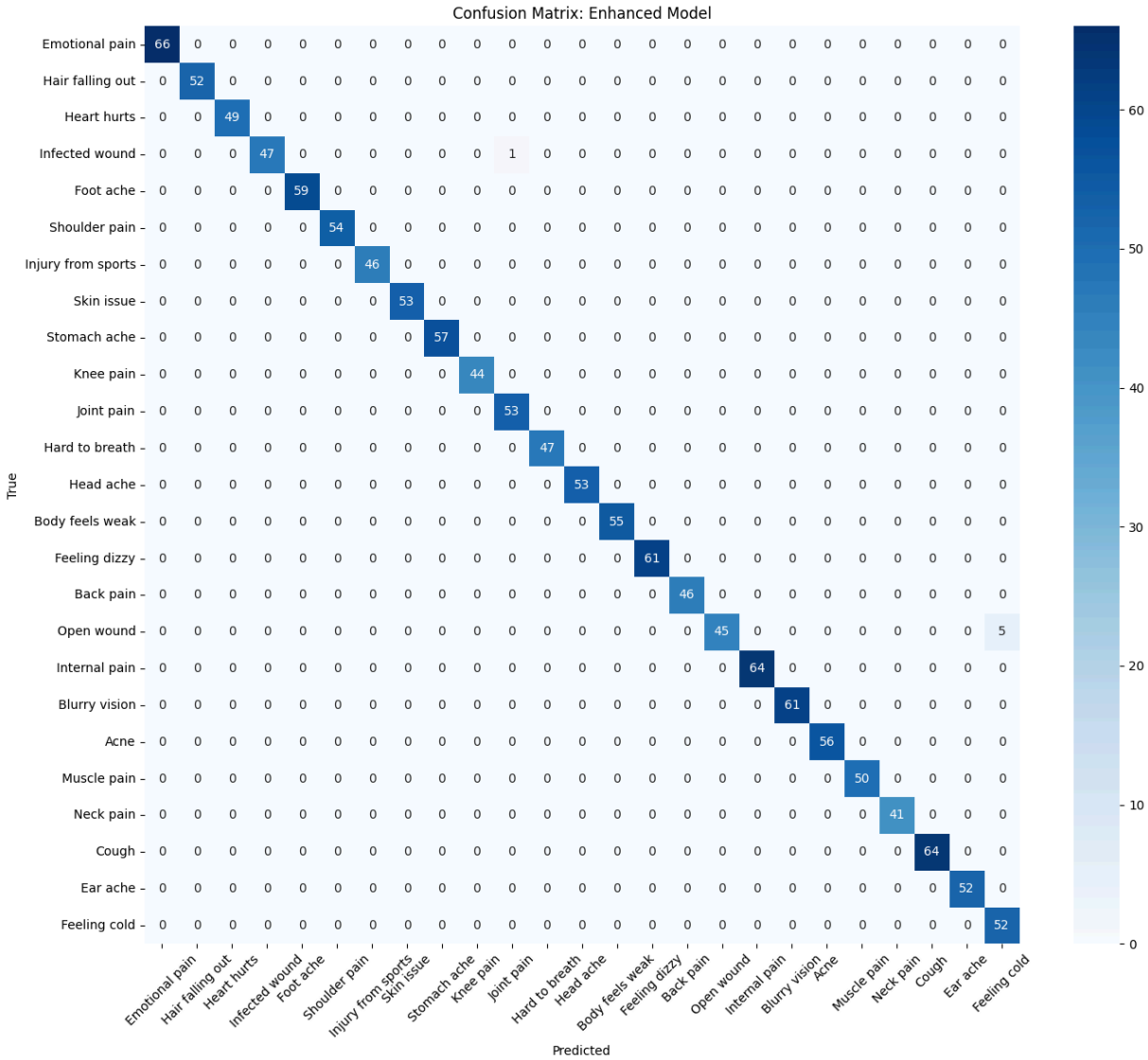
```
Analyzing feature importance...

Top features for 'Acne':
disease hair, gland, sebaceous, hair follicle, never, lot acne, whitehead, get, pimp
le back, patch, disease, break, face, pimple, acne

Top features for 'Back pain':
back extend, sit, back hurting, play sport, cant bend, lower back, burning, back hur
t, sensation spine, sport burning, spine, back pain, bend, pain back, back

Top features for 'Blurry vision':
fuzzy, glass, sterilizer eye, cloudy eye, cloudy, sterilizer, everything, see well,
cant see, thing, blurry vision, blurry, see, eye, vision

Top features for 'Body feels weak':
slowly, stand im, muscle feel, relief, strength, energy, dont, tried relief, relief
cant, feel tired, body feel, tired, feel weak, weak, body

Top features for 'Cough':
breathing night, ive, cant, stop coughing, mucus, breathe, congestion, congestion ch
est, feel congestion, pain throat, lung, cant breathe, coughing, throat, cough

Top features for 'Ear ache':
today pain, pain throbbing, throbbing unbearable, started tinnitus, im hard, hear, u
nbearable, cant hear, tinnitus, ear hurt, time hearing, pain ear, hearing, ear ache,
ear

Top features for 'Emotional pain':
breath fatigue, fatigue, heavy breath, mental suffering, mental, feel suicidal, suic
idal, think, remember, remember feel, pain think, feel, im disappointed, disappointe
d, sad

Top features for 'Feeling cold':
always, nose, even, get chill, chill ache, body, like refrigerator, refrigerator, te
mperature, running nose, chill, warm, feeling cold, feel cold, cold

Top features for 'Feeling dizzy':
like room, room spinning, room, spinning get, head spinning, felt dizzy, im going, r
ound, feel, stand, lightheaded, feel lightheaded, spinning, feel dizzy, dizzy

Top features for 'Foot ache':
stand touched, operation, nerve damaged, touched, nerve, really, walk, back leg, bac
k muscle, ache, heel, pain foot, foot hurt, foot ache, foot

Top features for 'Hair falling out':
soon sick, coming, hair coming, treatment, front, fall, scalp, itch front, center sc
alp, center, front center, lot hair, hair falling, falling, hair

Top features for 'Hard to breath':
great pressure, pressure chest, hardly, feel hard, something squeezing, squeezing lu
ng, morning respiration, respiration loud, respiration, cant take, hardly breathe, b
reathe, hard, chest, breath

Top features for 'Head ache':
drink lot, feel head, keep, head heavy, focus, headache every, brain, head ache, can
t keep, keep focus, skull, pain head, head hurt, headache, head
```

```
Top features for 'Heart hurts':
squeezing sensation, sensation chest, bear squeezing, saddens, chest saddens, inside
rib, strange powerful, ache left, breathe dull, left, pain chest, heart hurt, dia, c
hest, heart

Top features for 'Infected wound':
oozing, daughter, healing, look like, pu, infection, son, look, red, hand, person, i
njured person, wound, cut, infected

Top features for 'Injury from sports':
may, football, sprained, sprained ankle, got injured, playing soccer, hit, ankle, sl
eep cold, basketball, soccer, playing, match, injured, sport

Top features for 'Internal pain':
feel aching, hip, pain heavy, heavy breathing, hurt smoke, smoke, pain abdomen, side
ache, internal pain, abdomen, inside, pain body, feel sharp, pain, internal

Top features for 'Joint pain':
dont cry, elbow, pain joint, knee, joint pain, climb stair, climb, knee climb, ankle
hurting, bent, bent arm, hurt bent, hurting, ankle, joint

Top features for 'Knee pain':
go stair, walking, redness swelling, swelling difficulty, difficulty walking, pain k
nee, suddenly, difficulty, kneecap working, pain kneecap, knee hurt, kneecap, knee p
ain, bone, knee

Top features for 'Muscle pain':
shoulder muscle, muscle pain, sport pain, pain skin, im sport, pain, soreness body,
calve, pain calve, pain bicep, bicep, trapes, pain trapes, stretch, muscle

Top features for 'Neck pain':
look, moving, cant, left, move, neck pain, cant move, stiff, sore, neck hurt, pain n
eck, throught, throught sore, move head, neck

Top features for 'Open wound':
laceration rupture, laceration, opening, finger, open wound, cut im, im bleeding, op
ened, wound opened, thumb, bleeding, cut, heal, open, wound

Top features for 'Shoulder pain':
yesterday, try, shoulder hurt, pain shoulder, hurt raise, raise arm, side neck, pres
sure side, yesterday shouder, shouder pain, shouder, raise, arm, shoulder pain, shou
lder

Top features for 'Skin issue':
rash itch, itch bad, water, dark arm, something dark, fell hot, like fell, hot wate
r, arm, itchy, dark, red, itching, rash, skin

Top features for 'Stomach ache':
cramp along, meal, eat, abdominal, stomach eating, hard pain, feeling nauseous, naus
eous, stomach feel, abdominal pain, feel abdominal, stomach ache, eating, indigestio
n, stomach

Performing cross-validation for model robustness...

Logistic Regression cross-validation:
```

```
Mean CV Score: 0.9970, Std: 0.0017
Individual scores: [0.99699925 0.99474474 0.9954955  0.9984985  0.99924925]

Random Forest cross-validation:
Mean CV Score: 0.9962, Std: 0.0017
Individual scores: [0.99699925 0.99474474 0.9954955  0.99474474 0.99924925]

SVM cross-validation:
Mean CV Score: 0.9970, Std: 0.0017
Individual scores: [0.99699925 0.99474474 0.9954955  0.9984985  0.99924925]

Naive Bayes cross-validation:
Mean CV Score: 0.9929, Std: 0.0025
Individual scores: [0.99474869 0.99249249 0.98873874 0.99624625 0.99249249]
```



```
--- ENHANCED MODEL WITH ADDITIONAL FEATURES ---
Combined features shape: (5328, 3322)

Training enhanced Logistic Regression model with combined features...

Enhanced Model Results:
Accuracy: 0.9955
                    precision    recall  f1-score   support

              Acne       1.00      1.00      1.00        66
         Back pain       1.00      1.00      1.00        52
     Blurry vision       1.00      1.00      1.00        49
   Body feels weak       1.00      0.98      0.99        48
             Cough       1.00      1.00      1.00        59
          Ear ache       1.00      1.00      1.00        54
    Emotional pain       1.00      1.00      1.00        46
      Feeling cold       1.00      1.00      1.00        53
     Feeling dizzy       1.00      1.00      1.00        57
         Foot ache       1.00      1.00      1.00        44
  Hair falling out       0.98      1.00      0.99        53
    Hard to breath       1.00      1.00      1.00        47
         Head ache       1.00      1.00      1.00        53
       Heart hurts       1.00      1.00      1.00        55
    Infected wound       1.00      1.00      1.00        61
Injury from sports       1.00      1.00      1.00        46
     Internal pain       1.00      0.90      0.95        50
        Joint pain       1.00      1.00      1.00        64
         Knee pain       1.00      1.00      1.00        61
       Muscle pain       1.00      1.00      1.00        56
         Neck pain       1.00      1.00      1.00        50
        Open wound       1.00      1.00      1.00        41
     Shoulder pain       1.00      1.00      1.00        64
        Skin issue       1.00      1.00      1.00        52
      Stomach ache       0.91      1.00      0.95        52

          accuracy                           1.00      1333
         macro avg       1.00      1.00      1.00      1333
      weighted avg       1.00      1.00      1.00      1333


Comparison:
Best TF-IDF model (Logistic Regression): 0.9955
Enhanced model with additional features: 0.9955
Improvement: 0.0000 (0.00%)
```




```
Saving the best model...
Model saved as 'models/text_classification_model.pkl'

--- SUMMARY AND CONCLUSIONS ---
1. The text classification analysis was performed on multiple medical prompt categor
ies
2. The best performing model was Logistic Regression with accuracy: 0.9955
3. Cross-validation showed a robust performance with mean accuracy: 0.9970
4. Incorporating additional features did not significantly improve performance
5. The trained model can effectively classify medical prompts based on text transcri
ptions
6. Key findings from the confusion matrix show which prompt categories are more easi
ly confused
7. Research objective of performing multi-class classification of medical prompts ha
s been achieved
```